Wellness Studio OS — Solution Architecture Document

Document 2 of 6 · Version 1.0 · 29 September 2026 Primary source: Master PRD v1.0 (and its Appendix A conflict register). Scope of this document: architecture blueprint only — no requirements restatement, database schemas, API contracts or project plans.

Assumptions carried from the PRD: ERPNext/Frappe is the ERP (A-01); a single NestJS API layer sits between the front ends and the ERP (A-02, pending mentor confirmation); Phase 1 is a 5-day delivery with WhatsApp and Razorpay in test/mock mode (A-05); a commercial foundation model is used in Phase 1 (A-06).

---

1. Architecture Overview

1.1 Architectural Goals

#	Goal	Measure
G1	Deliver a working multi-tenant operational core in 5 days	Canonical flow passes E2E: Member → Membership → Class → Booking → Attendance → Payment → Notification → Dashboard
G2	One governed capability layer for UI, AI and automation	Zero business logic duplicated outside the application/ERP layers
G3	Tenant isolation from day one	Cross-tenant access denied and logged; isolation tests pass
G4	AI-ready without redesign	Agents added as consumers of existing capabilities
G5	Replaceable edges (ERP, LLM, messaging, payment)	Each behind an adapter interface
G6	Failure-tolerant side effects	Messaging/workflow outage never blocks or reverses a transaction
G7	Traceability	Correlation ID links UI → API → ERP → n8n → WhatsApp

1.2 Design Principles

1. **ERP is truth, AI assists.** Payments, memberships, bookings and attendance are deterministic logic; AI drafts, summarises and recommends.
2. **One front door.** Browsers, AI, n8n and agents never touch the ERP or database directly; every request passes the API layer.
3. **Same capability, every channel.** The Assistant calls the same application services the UI calls — identical validation, permissions and audit.
4. **Configure, don't hard-code.** Plans, class types, capacity, templates, timings and branding are tenant data.
5. **Commit first, notify after.** Events are emitted only after the transaction commits; consumers are idempotent.
6. **Modular monolith first.** One deployable API with strict internal module boundaries; extract services only when load demands.
7. **Least privilege.** Restricted ERP integration identity; agents get only their domain's tools; AI inherits the invoking user's permissions.
8. **Fail visibly, never falsely.** Unknown state is reported as pending/failed — never guessed success.

1.3 Key Architectural Decisions

ID	Decision	Rationale	Trade-off / Note
AD-01	ERPNext on Frappe, hosted on Frappe Cloud	Team decision; fastest ERP path for 5-day delivery	Managed hosting constrains OS-level customisation; confirm custom-app support early (R-14)
AD-02	ERPNext core untouched; all product logic in a custom Frappe app	Upgradeability	Custom app owns rules that must be transactional (capacity, uniqueness)
AD-03	Single NestJS API layer between Next.js and ERP	One place for auth, tenancy, RBAC, validation, audit, AI orchestration	Extra hop (R-03) — mitigated by thin ERP adapter and contract-first interfaces. **Pending mentor confirmation**
AD-04	Modular monolith (NestJS)	Small team, 5-day window	Extract AI/notification workers later
AD-05	PostgreSQL for application data; ERPNext keeps its default DB	Avoids ERP-on-PostgreSQL maturity risk	Two data stores → explicit ownership rules (§7)
AD-06	Transactional outbox for events; n8n consumes signed webhooks	Reliable, replayable, commit-before-notify	Small dispatcher worker required
AD-07	n8n orchestrates, WhatsApp Cloud API delivers, n8n is never a source of truth	PRD BR-NTF-01	Business state lives in the application/ERP, not workflow variables
AD-08	AI layer is a module inside the API layer in Phase 1	Direct reuse of services, permissions and audit	Separable to its own service later behind the same tool interface
AD-09	Two-phase confirmation for AI actions (propose → confirm)	No false success; args cannot drift after user confirms	Pending-action store in PostgreSQL
AD-10	One Next.js app, two route groups (staff, member) with shared design system	Fastest delivery; shared components	Split into two apps if release cadence diverges
AD-11	Tenant resolved from host (subdomain) and bound to session	Simple, extensible to custom domains	Wildcard DNS/TLS needed
AD-12	Model-agnostic LLM gateway	Avoid provider lock-in (R-10)	Prompt/tool-format normalisation layer

1.4 Technology Overview

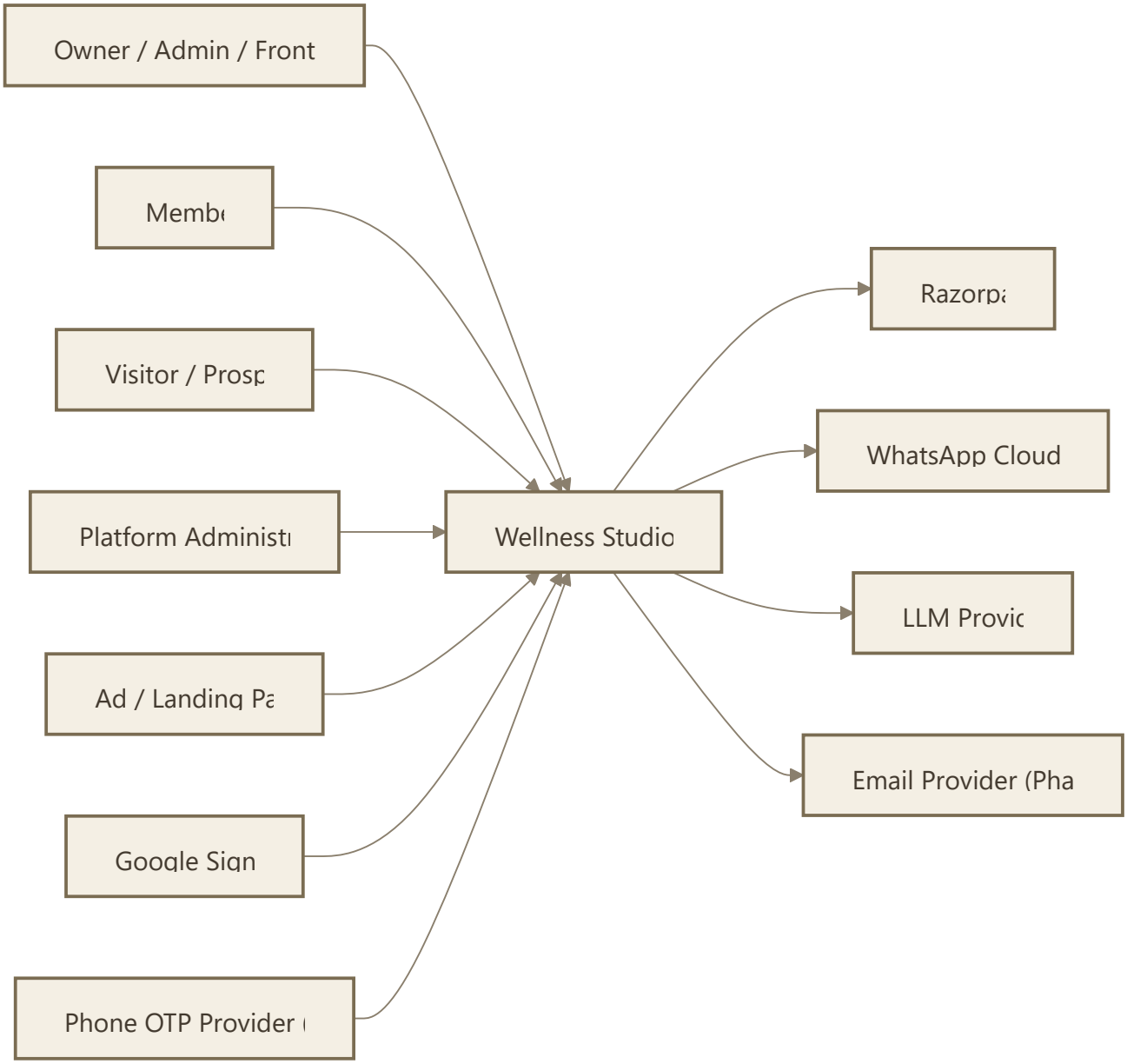
Layer	Technology	Hosting
Web (B2B + B2C)	Next.js + TypeScript, Tailwind, shadcn/ui	Oracle Cloud

Layer	Technology	Hosting
API / business layer	NestJS (modular monolith), OpenAPI-first	Oracle Cloud
Application DB	PostgreSQL (ORM: Prisma per rebuild plan)	Oracle Cloud
ERP	ERPNext on Frappe + custom app	Frappe Cloud
Workflow engine	n8n (self-hosted)	Oracle Cloud
Messaging	WhatsApp Cloud API (mock mode in Phase 1)	Meta
Payments	Razorpay (test mode in Phase 1)	Razorpay
AI	Commercial LLM via API (provider TBD) behind a gateway	External
Email (Phase 2)	Transactional email provider (TBD)	External
Optional infra	Redis (cache/queue) and object storage — carried from earlier stacks but not reconfirmed by the PRD; adopt only if needed	Oracle Cloud
Source	Private GitHub monorepo + GitHub Actions	GitHub

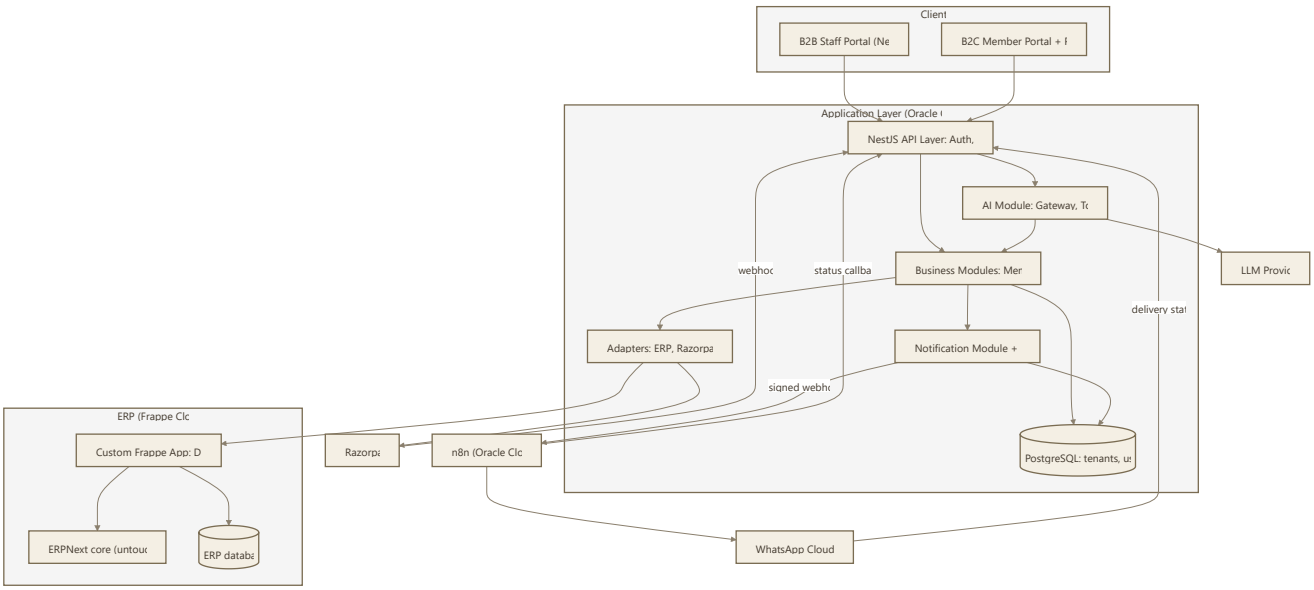
---

2. High-Level Architecture

2.1 System Context



2.2 High-Level Architecture

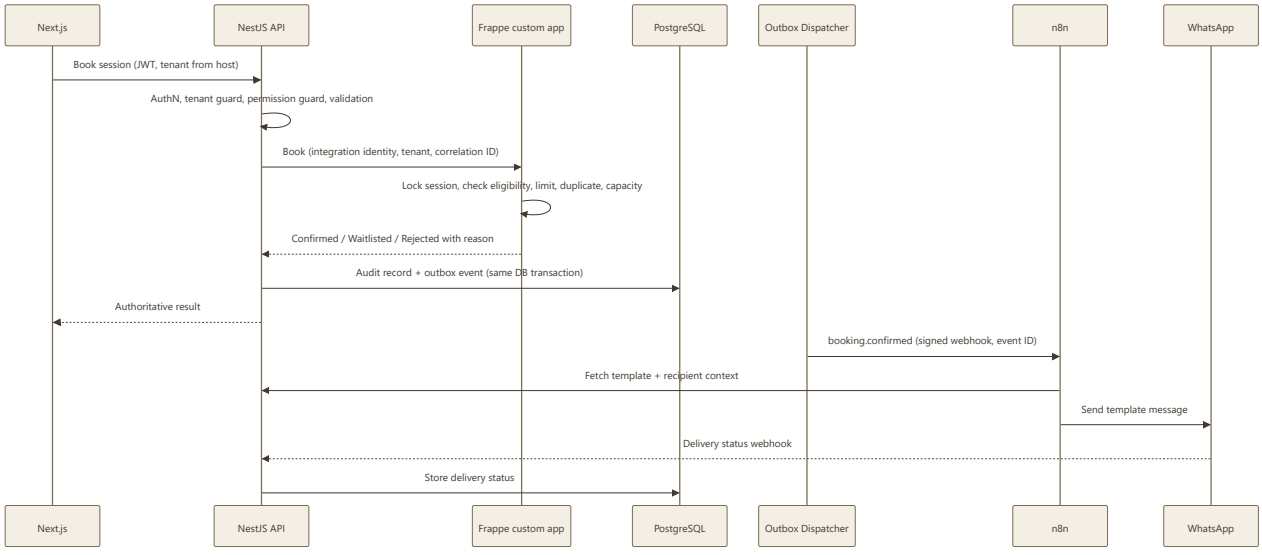


2.3 Major Components

Component	Responsibility	Never does
Next.js web	Rendering, forms, client state, route protection UX	Decide business state; call ERP; hold secrets
API layer (NestJS)	AuthN/Z, tenancy, validation, orchestration of business capabilities, audit, event emission	Bypass ERP for ERP-owned data
Business modules	Domain services shared by REST controllers and AI tools	Depend on UI concerns
ERP adapter	Translate application intents to Frappe calls; map errors; enforce timeouts/retries	Leak ERP internals to callers
Custom Frappe app	ERP-side validation, atomic operations (capacity, uniqueness), DocTypes, permission hooks	Modify ERPNext core
PostgreSQL	Tenant, identity, configuration, audit, AI, notification, outbox data	Store ERP-owned business records as a second truth
AI module	Intent → tool selection, confirmation, response generation, AI audit	Touch DB/ERP directly
Outbox dispatcher	Deliver committed events to n8n with retry and dedup	Decide message content
n8n	Multi-step orchestration, schedules, retries, WhatsApp/email calls	Own business rules or state

2.4 Data Flow Overview

Canonical write path (booking example):



Read path: UI/AI → API (permission + tenant scope) → ERP for business data, or PostgreSQL for app data → shaped response. Dashboards and the Assistant use the **same reporting service**, guaranteeing identical figures.

---

3. Application Architecture

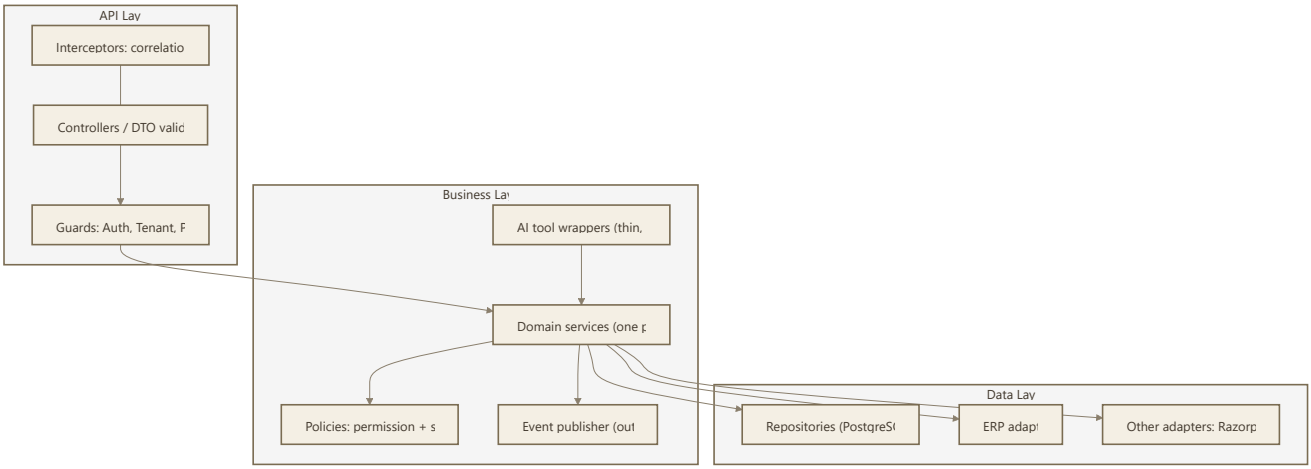
3.1 Frontend

One Next.js codebase, two portals via route groups, one shared component library.

Aspect	B2B Portal (staff)	B2C Portal (member + public)
Users	Owner, Admin, Front Desk, Trainer	Member, Visitor
Auth	Email + password (MFA before prod)	Phone OTP, optional Google; public routes need none
Layout	Sidebar dashboard, tables, forms, role-aware navigation	Mobile-first, bottom-nav style, minimal steps
Rendering	Client-heavy dashboards with server-fetched data	SSR/ISR for public class and event pages (SEO); client for member actions
Tenant	Resolved from host; branding injected as theme tokens	Same
AI surfaces	Admin Assistant panel	Enquiry chat (public), booking suggestions, recommendations

Frontend rules: feature-folder structure; typed API client generated from the OpenAPI contract; permission-aware UI (hide, never trust — server enforces); every screen has loading/empty/error/permission-denied states; no business rules in components; no secrets in client code; tenant theme (name, logo, colours) loaded at layout level.

3.2 Backend — Three Layers Inside NestJS



Layer	Responsibilities	Rules
API layer	HTTP/webhook entry, authentication, tenant binding, permission checks, input validation, error normalisation, correlation IDs	Thin. No business logic. Versioned (`/api/v1`), contract-first
Business layer	Domain services implementing capabilities; orchestration across ERP and PostgreSQL; event emission; AI tool wrappers	The only place business decisions are composed. Controllers and AI tools call the *same* services
Data layer	Repositories for PostgreSQL; ERP adapter; third-party adapters	Every repository call is tenant-scoped; adapters own retries, timeouts, error mapping, idempotency keys

Interaction contract: Controller → Guard chain → Service → (Policy check → ERP adapter and/or Repository → Outbox write) → Interceptor writes audit → response. AI tools enter at **Service**, skipping HTTP but not the guards' outputs (the AI request context carries the same user, role, tenant and permissions).

---

4. B2B Architecture

Common to all B2B modules: tenant-scoped, permission-checked, audit-logged, four UI states. Data of record is the ERP unless stated as "App DB".

Module	Purpose	Components	Dependencies	Data Flow
Dashboard	Role-aware KPI view and quick actions	KPI tiles UI; Reporting service; role-visibility policy	Members, Memberships, Attendance, Payments, Events services	UI → Reporting service → aggregates from ERP (cached briefly) → tile payload. Same service serves the Assistant
CRM	Lead/registrant capture to conversion	Lead list/detail UI; public form endpoint; CRM service; conversion orchestrator; duplicate detector	Members service (conversion uses standard member creation), Notification, ERP	Form/ad → public endpoint (rate-limited, tenant from host) → CRM service → ERP lead record → outbox `lead.created` → n8n confirmation. Conversion → Members service → member created, lead source/history retained
Members	Member lifecycle, one record per person	Member list/form/profile UI; Members service; duplicate check; archive	ERP (system of record), Audit	UI → API → duplicate check (email/phone) in ERP → create/update → audit before/after. IDs generated by ERP
Memberships	Plans, assignment, approval, expiry, manual renewal	Assign/approve UI; Memberships service; approval workflow (Frappe); expiry scheduler	Members, Payments (independent state), Config (plans), Notification	Assign → Pending Approval → (payment verified, approval granted, independent) → Active → scheduler marks Expired → outbox `membership.expiring/expired`
Classes	Class types, sessions, trainer, capacity	Class-type & session UI; Classes service; recurrence expander	Config (types, default capacity), Staff (trainer)	Config supplies type/capacity → session records in ERP; capacity reduction checked against confirmed bookings before commit
Bookings	Reserve seats with eligibility/capacity control	Booking UI (self/on-behalf); Booking service; **ERP atomic booking method**; waitlist promoter	Memberships (eligibility), Classes (capacity), Payments (if	Request → policy check → ERP locks session, validates, commits confirm/waitlist/reject → outbox event. Cancel outside cutoff → seat freed → promote

Module	Purpose	Components	Dependencies	Data Flow
			paid), Notification	next eligible → outbox `waitlist.promoted`
Attendance	Auditable per-session attendance	Roster UI; QR generator/validator; trainer marking UI; correction form; Attendance service	Bookings, Classes, Events, Staff (trainer assignment)	QR/trainer/front-desk → validate session, eligibility, duplicate → ERP record. Correction → immutable audit (actor, time, previous, new, reason)
Events	Workshops and studio events	Event editor; registration UI; Events service; payment-hold timer	Payments, CRM (external registrants become leads/registrants), Notification, Config (categories)	Registration → seat/window/audience check → free: confirm; paid: hold → verified payment → confirm; expired hold releases seat
Payments	Collect, record, reconcile	Payment list UI; Payments service; Razorpay adapter; webhook receiver; manual-record form	Memberships, Events, Bookings, ERP, Razorpay	Initiate → gateway order → client pays → Razorpay webhook (signature verified, idempotent) → payment Paid → dependent state updated → outbox `payment.confirmed`. Delayed webhook leaves Pending
Reports	Operational and financial insight	Report views; Reporting service; query definitions	All read models	UI/Assistant → Reporting service → ERP aggregates → permission-filtered result. Phase 2 exports use the same filters
Staff Management	Staff, roles, scope, trainer assignment, grants	Staff UI; Staff service; permission-grant editor	Auth/identity (App DB), ERP (trainer records)	Owner edits role/grant → App DB (roles/permissions) → effective next request → audit. Deactivation blocks login, history retained
Settings (Configuration Center)	Branding, plans, class types, capacity, cancellation window, categories, templates, timings, feature flags	Settings UI per area; Config service; config cache	Everything (consumed by all modules)	Owner edits → App DB (branding, templates, timings, flags) or ERP (plans, class types) → audit → cache invalidation → effective without deploy

Module	Purpose	Components	Dependencies	Data Flow
Notifications	Configurable WhatsApp workflows and delivery visibility	Template & timing editor; delivery log UI; Notification service; outbox dispatcher	n8n, WhatsApp adapter, Config, Audit	Committed event → outbox → n8n → WhatsApp → status callback → stored delivery record → admin visibility
AI Assistant	Natural-language queries and Create Member	Chat panel; AI orchestrator; tool registry; confirmation card; AI audit	Reporting, Members, Memberships, Attendance services; LLM gateway	See §8.1

---

5. B2C Architecture

Members act only on **own records**; the API derives the member identity from the session, never from client-supplied IDs.

Module	Purpose	Components	Dependencies	Data Flow
Dashboard	Status, upcoming bookings, renewal prompt	Dashboard UI; Member summary service	Memberships, Bookings, Attendance, Events	UI → summary service (own member scope) → ERP → payload; renewal CTA if ≤7 days to expiry
Membership	Browse, buy, renew	Plan catalogue UI; purchase flow; Memberships & Payments services	Config (active plans), Payments, Approval	Select plan → Pending membership → checkout → verified payment → approval if required → Active
Bookings	Live availability, book, cancel, waitlist	Schedule UI; availability service; booking flow	Classes, Memberships (eligibility reason), Bookings service	Schedule fetch shows eligibility reason per session → book → authoritative ERP result (never cached state) → confirmation
Attendance	QR check-in and history	QR scan UI; history view	Attendance service, Bookings	Scan → validate session/eligibility/duplicate → record; history includes admin corrections
Events	Discover, register, pay	Event list (audience-filtered); registration flow	Events, Payments	Members-only events hidden from non-members; paid registration confirmed only after verified payment

Module	Purpose	Components	Dependencies	Data Flow
Payments	Checkout and history	Checkout UI (Razorpay); history list	Payments service	Own payments only; pending/failed never presented as success
Profile	Own details, OTP/Google login, linking	Profile form; login flows; account-linking service	Identity (App DB), Members	Edits limited to permitted fields; login method change re-verifies identity; linking matches on verified phone to prevent duplicates
Notifications	WhatsApp confirmations/reminders (in-app centre Phase 2)	Preference view (Phase 2)	Notification pipeline (§10)	Passive: receives messages produced by automations
Recommendations	Rules-constrained class suggestions	Recommendation panel; Recommendation service	Classes, Bookings, Memberships (entitlement), AI gateway (optional explanation)	Rules produce eligible+available candidate set → ranking/explanation (rules first, LLM wording optional) → booking via standard flow
Public site	Discovery, enquiry, workshop registration	Public pages (SSR); Enquiry chat; registration form	Classes, Events, CRM, AI Enquiry	Anonymous visitor scoped by host tenant; only public data and lead creation

---

6. Multi-Tenant Architecture

6.1 Tenant Isolation Strategy

Hierarchy: **Tenant** → **Studio** → **Branch**. All business and application records carry this scope from day one.

Layer	Isolation mechanism
Request	Tenant resolved from host; JWT carries tenant and membership; tenant guard rejects mismatch; tenant context held in request-scoped storage
API/service	Every service and repository takes tenant context implicitly; no unscoped query path exists in code
PostgreSQL	Tenant column on every table + row-level security keyed to a per-request session setting (defense in depth)
ERP	**Recommended (to validate in the Day-1 spike):** single Frappe site for beta; each tenant is an ERPNext *Company*; every custom DocType carries a tenant link; permission-query hooks in the custom app restrict all reads/writes to the tenant passed by the integration call; the adapter always injects tenant

Layer	Isolation mechanism
AI	Tool calls run under the invoking user's tenant; prompts and retrieval sources are tenant-filtered; no cross-tenant retrieval index
Automation	Every event carries tenant ID; n8n workflows resolve tenant configuration by ID; templates and credentials are per tenant
Files/logs	Tenant-prefixed storage paths; logs tagged with tenant and correlation ID

Verification: automated cross-tenant tests (every module) run in CI; denied attempts are audit-logged (BR-GEN-02).

6.2 Data Ownership

Data	Owner	Notes
Members, memberships, member plans, class types, sessions, bookings, attendance, events & registrations, payment records, invoices, leads	**ERP**	Business truth
Tenants, studios, branches, SaaS tier & feature flags	**App DB**	Platform data
Users, credentials, roles, permissions, grants, sessions, OTP/Google links	**App DB**	Identity; members are *not* ERP users
Branding, notification templates, reminder timings, cancellation window, default capacity	**App DB**	Tenant configuration
Audit log (incl. AI), pending AI actions, AI conversations (temporary)	**App DB**	Immutable audit
Notification records, delivery status, outbox events	**App DB**	Operational
Workflow definitions	**Repo (n8n exports)**	Versioned as code

6.3 Tenant Configuration

Configuration is resolved as: **platform defaults → tier entitlements → tenant overrides**. Examples: default capacity 30, cancellation window 4 hours, reminder schedule, feature flags gating AI/automation by tier (BR-AI-11). Configuration is read through a cached Config service; changes are audited and invalidate the cache. Business-record configuration (plans, class types) stays in the ERP; presentation/operational configuration stays in the App DB. A single onboarding routine (seeded/assisted in Phase 1) creates: tenant → first studio → first branch → Owner user → default configuration.

6.4 Future Scaling Strategy

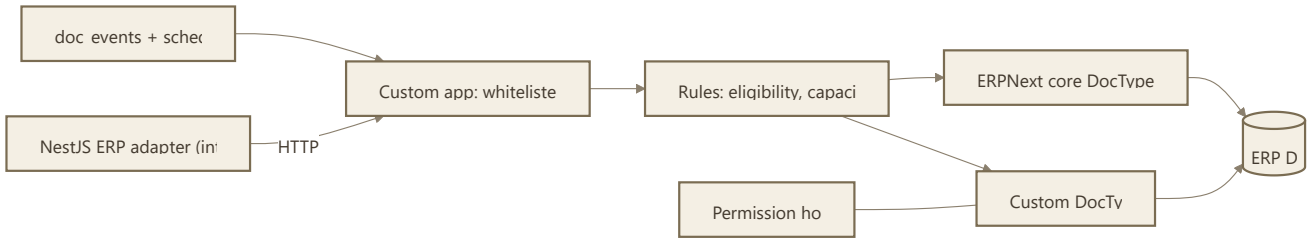
Trigger	Step
More tenants on one ERP site	Add ERP read-through caching for dashboards; move heavy reports to scheduled aggregates
Noisy tenant / enterprise customer	Site-per-tenant (dedicated Frappe site) for that tenant; ERP adapter routes by tenant → site map (no caller changes)
Multi-branch / franchise	Branch scope already present; add location-aware roles and reporting

Trigger	Step
White-label	Custom domains map to tenant; theme tokens already tenant-configurable
API load	API is stateless → horizontal scale behind load balancer; add Redis for cache/queues
AI/notification load	Extract AI module and outbox dispatcher into separate workers
PostgreSQL growth	Managed PostgreSQL with read replica; partition audit and notification tables by time

---

7. ERP Architecture

7.1 ERPNext/Frappe Architecture



- **Two Frappe artefacts only:** ERPNext (upstream, unmodified) and the custom app (private repo, installed on the Frappe Cloud bench).
- **Access:** the adapter uses a restricted API identity; the custom app exposes purpose-built whitelisted methods for write operations that must be atomic (booking, waitlist promotion, check-in, membership activation) and standard resource access for simple reads.
- **Why methods for writes:** capacity, duplicate and eligibility checks must run in one ERP transaction with row locking so last-seat contention yields exactly one confirmation (NFR-PRF-03).
- **ERPNext core reuse:** use ERPNext accounting/invoicing/customer structures where they fit (Phase 2 GST/invoicing); wellness-specific entities are custom DocTypes. The precise DocType mapping is a Day-1 design task and is not fixed here.

7.2 What Belongs Where

ERP (custom app + ERPNext)	Application layer (NestJS + PostgreSQL)
Atomic, transactional business rules (capacity, duplicates, eligibility, membership state transitions)	Authentication, sessions, OTP/Google, MFA
System-of-record entities (see §6.2)	Tenant/studio/branch model, tier flags, feature entitlement
Approval workflows and expiry scheduler	RBAC/permission evaluation and grants
Invoicing, accounting, GST (Phase 2)	Orchestration across ERP + payments + notifications
ERP-native reports/aggregates	Reporting service shaping, dashboards, exports
Immutable attendance-correction history	Cross-cutting audit log, AI audit

ERP (custom app + ERPNext)	Application layer (NestJS + PostgreSQL)
—	Configuration, templates, notification records, outbox
—	AI orchestration, pending actions

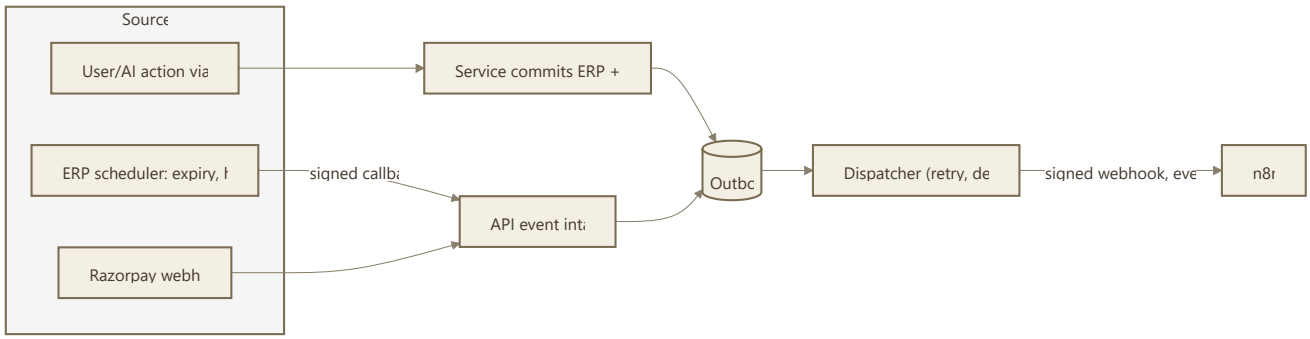
Rule of thumb: if losing it would corrupt a business fact, it lives in the ERP; if it governs *who may do what, how it is delivered, or how it is presented*, it lives in the application layer.

7.3 Synchronization Strategy

Because ERP is the single record for business entities, there is **no bidirectional entity sync**. Instead:

Pattern	Use
Synchronous call-through	All ERP-owned reads/writes go through the adapter at request time. No shadow copy of business truth
Reference by ID	App DB stores ERP document IDs (e.g., audit entries, notifications, pending actions) — never copies of business fields beyond what is needed for display/audit snapshots
Short-TTL read cache (optional)	Dashboard aggregates and configuration only; never for booking availability or eligibility
Identity mapping	App DB maps user identity ↔ member/staff ERP ID; account linking resolves on verified phone to avoid duplicates
Reconciliation job (nightly)	Detects orphaned references (ERP record missing vs. App DB reference) and outbox events without delivery outcome; reports to admins

7.4 Event Flow



- Application-originated events: written to the outbox **in the same PostgreSQL transaction** as the audit record, after the ERP call has succeeded.
- ERP-originated events (expiry, timeouts): ERP scheduler posts a signed callback to the API, which validates, audits and writes to the outbox — so all events share one path.
- Time-based reminders (class, event, renewal): n8n schedules trigger a "what is due" query on the API; the API returns tenant-scoped, deduplicated recipients.
- Event envelope (conceptual): event ID, type, tenant, occurred-at, subject reference, correlation ID. Consumers are idempotent on event ID.

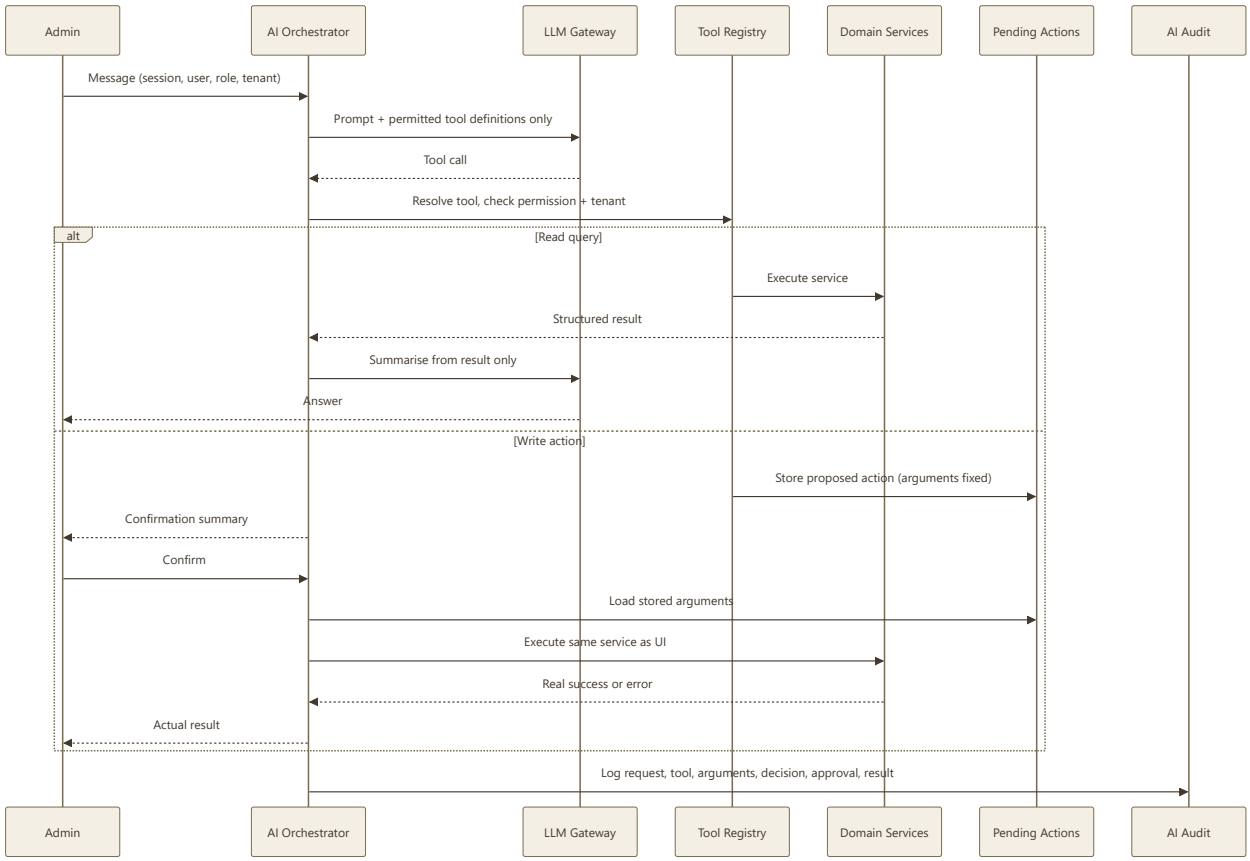
7.5 Integration Points (ERP)

Point	Direction	Notes
Adapter → custom app methods/resources	NestJS → ERP	Restricted identity, timeouts, retry on safe reads only, idempotency key on writes
ERP scheduler/hooks → API event intake	ERP → NestJS	Signed callbacks
Payment posting to ERP	NestJS → ERP	After verified Razorpay confirmation
Report aggregates	NestJS → ERP	Used by dashboards and Assistant
ERP unavailable	—	Adapter returns explicit "temporarily unavailable"; UI shows clear failure; nothing cached is presented as confirmed (NFR-AVL-03)

---

8. AI Architecture

8.1 Shared AI Platform (inside the NestJS AI module)



Platform components

Component	Responsibility
LLM Gateway	Provider-agnostic interface (tool-calling, streaming, timeouts, retries); prompt templates versioned in the repo; provider selectable per environment

Component	Responsibility
Tool Registry	Each tool = name, purpose, argument schema, required permission, risk class (read / write), owning service. Tools exposed to the model are filtered per user, tenant, tier and feature flag
Guardrails	System prompt policy; scope refusal for unsupported questions; no medical/health advice; PII minimisation before prompts and logs; output must derive from tool results
Pending Action Store	Holds proposed write actions with fixed arguments, requester, expiry and status; only the stored arguments execute after confirmation
Conversation Store	Temporary context, retained short-term; never becomes business data except through a governed service
AI Audit	One immutable record per model-initiated tool call (NFR-AUD-03)
Evaluation Harness	Curated query set and Create Member scenarios run in CI; results recorded (supports 85% / 95% / zero-false-success targets)

8.2 Assistants

AI Admin Assistant

Item	Detail
Purpose	Natural-language operational queries and Create Member with confirmation
Inputs	Admin message; user identity, role, tenant; conversation context
Outputs	Concise answer; confirmation card; actual action result with record ID or precise error; audit record
Data sources	Reporting service (expiring memberships, low attendance) and Members service — the same ones behind dashboards and UI
Integration points	Staff portal chat panel; LLM gateway; tool registry; pending-action store; audit. Relative dates ("Monday") resolved by the service layer and shown in the confirmation
Notes	Missing mandatory fields → ask, never infer. Duplicate/validation errors from the service are relayed verbatim. Front Desk: read tools only, when enabled

AI Enquiry Assistant

Item	Detail
Purpose	First-line answers for prospects; lead capture
Inputs	Visitor question; tenant from host
Outputs	Grounded answer; lead record after consent; handoff/escalation message
Data sources	Public class/schedule/plan/event data (read via public-scope services); tenant-approved FAQ content in App DB
Integration points	Public site chat widget; CRM service (create lead); Notification pipeline (confirmation). Runs under a restricted *visitor principal*: public-read tools plus create-lead only; rate-limited and

Item	Detail
	bot-protected
Notes	No invented prices or promises; escalate to human when confidence or source coverage is insufficient

AI Booking Assistant

Item	Detail
Purpose	Shortlist eligible, available sessions and hand off to standard booking
Inputs	Member preference (type, day, time); member identity
Outputs	Ranked shortlist with reasons and deep links to booking flow
Data sources	Classes, Bookings (live availability), Memberships (eligibility) services
Integration points	Member portal chat; eligibility-check tool (read-only). No booking tool in Phase 1 — booking always occurs in the standard flow (in-chat booking Phase 2 using propose/confirm)

AI Recommendation Assistant

Item	Detail
Purpose	Explainable class suggestions
Inputs	Stated goal and level; attendance history; plan entitlement
Outputs	Ranked suggestions with short reasons
Data sources	Attendance, Classes, Memberships services
Integration points	Recommendations panel. **Rules select the candidate set (eligible + available); the LLM only ranks/explains within it**, so ineligible classes can never be suggested. No health claims

AI Feedback Assistant

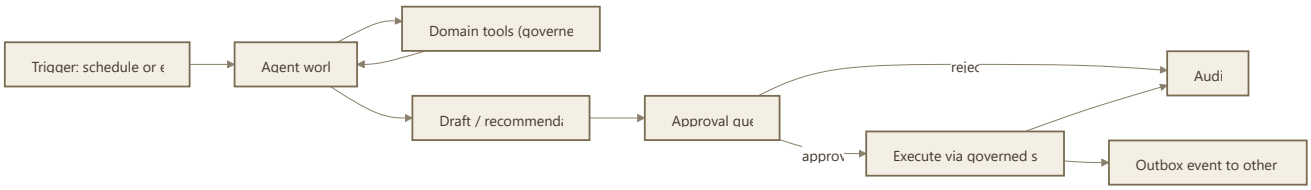
Item	Detail
Purpose	Convert free-text comments to structured insight
Inputs	Comment with class/session reference; optional rating
Outputs	Sentiment, topic, key point; weekly theme summary
Data sources	Feedback records (App DB, referencing ERP session IDs)
Integration points	Post-session feedback form → async classification job → admin feedback view. Raw comment remains the record; results are advisory and trigger no automatic member action

---

9. Future Agent Architecture

Common agent runtime (Phase 2+): agents run as background workers using the same tool registry, LLM gateway, audit and pending-action store as the Assistant. Each agent has a **service principal scoped to one**

tenant and one domain toolset; when acting for a user, the user's permissions apply as a ceiling. Agents propose; the approval queue (new UI in Phase 2) records the human decision. Cross-agent collaboration happens through workflow events (outbox) rather than agent-to-agent calls.



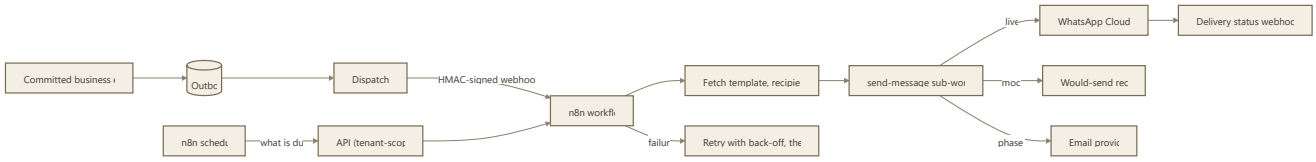
Agent (Phase)	Goal	Workflow	Data sources	Human approval
CRM Agent (P2)	Reduce lead leakage; lift conversion	On `lead.created` / stale-lead schedule → score and qualify → recommend next action → draft follow-up	Leads, sources, follow-up history, conversion events	Required before any message is sent; scoring/recommendations are advisory
Operations Agent (P2)	Continuous operational monitoring	Scheduled scans of capacity, attendance, bookings, expiries → raise alerts/tasks → propose schedule adjustments	Classes, Bookings, Attendance, Memberships	Alerts need none; any schedule change or member-facing action needs approval
Marketing Agent (P3)	Retention, re-engagement, promotion	Segment audience → draft campaign (WhatsApp/email) → submit for approval → hand to n8n → summarise performance	Member segments, renewal & activity data, event calendar, consent/opt-in flags	Mandatory before every send; opt-outs enforced by the platform, not the agent
Advertising Agent (P3)	Efficient acquisition	Analyse lead-source performance → draft ad copy and audiences → recommend landing pages	Lead source & conversion data, offers, events	**Mandatory before any publication or spend**; no platform credentials with publish rights are given to the agent
Growth Optimization Agent (P3)	Revenue and utilisation	Analyse revenue, occupancy, conversion → ranked, explained recommendations	Revenue, occupancy, conversion, plan data	Advisory only; owner acts through normal UI
Studio Insights Agent (P3)	Proactive intelligence and forecasts	Periodic briefing generation; anomaly detection; forecasts when data matures	Cross-module operational and financial data	Read-only; briefings delivered without approval; any action derived goes through other agents' queues
Session Summary	Readable progress and	Post-session/period job → generate	Attendance, session data,	Trainer/admin review before member-facing delivery is

Agent (Phase)	Goal	Workflow	Data sources	Human approval
Agent (Future)	session summaries	member and staff summaries from approved data	approved trainer notes	recommended by default

---

10. Automation Architecture

10.1 Pattern



Shared rules

- **Idempotency:** key = event ID + recipient + template; the API refuses duplicate sends.
- **Consent/opt-in:** checked in the API before a message is released to the send sub-workflow.
- **Mock mode:** environment switch inside the single `send-message` sub-workflow; live credentials replace mock via configuration only. Mock writes a would-send record (recipient, template, time).
- **Failure handling:** bounded retries with back-off → status `failed` stored → admin task/alert. A failed message never reverses the transaction.
- **Config-driven:** templates, timings and enablement come from tenant configuration, not workflow JSON.
- **Repo-managed:** workflows exported as JSON, reviewed and deployed with the release.

10.2 Automations

Automation	Trigger	Flow	Channel	Phase
Lead	Landing/ad form → `lead.created`	Confirmation immediately → follow-up sequence (P2) → conversion event stops sequence	WhatsApp (email P2)	Confirmation P1; sequences P2
Booking	`booking.confirmed`, `booking.cancelled`, `waitlist.promoted`	Immediate confirmation → class reminders 12h and 1h before (schedule) → promotion notice (P2)	WhatsApp	Confirmation + reminders P1
Membership renewal	Schedule scanning expiries; `membership.activated`	Activation notice → reminders at 7/3/1 days → expiry-day notice → status check and escalation to admin (P2)	WhatsApp (email P2)	Reminders P1
Attendance	`attendance.recorded`, absence detection (P2)	Optional post-class note or feedback request; inactivity nudges at 3 and 7 days (P2)	WhatsApp	P2 (except feedback link optional)

Automation	Trigger	Flow	Channel	Phase
Payment	`payment.confirmed`, `payment.failed`	Payment-received message → triggers membership-activated notice if approval already granted; failure → admin exception task	WhatsApp; email receipts P2	P1
Event	`event.registered`, schedule	Registration confirmation immediately → reminders 24h/6h/1h (workshops 24h/12h/1h)	WhatsApp	Confirmation P1; reminders P2
Notification (platform)	Any of the above	Central `send-message` sub-workflow: template render → consent check → send/mock → delivery status → retry/alert	WhatsApp, email (P2)	P1

Email flows (Phase 2): same pattern, alternate branch in `send-message`; templates managed in the Configuration Center; transactional emails (receipts, renewals) first, campaign email only via approved Marketing Agent drafts. Provider is not yet chosen.

---

11. Integration Architecture

Integration	Purpose	Data Flow	Error Handling
ERPNext	System of record for core business data	NestJS ERP adapter → Frappe (purpose-built methods/resources) and ERP callbacks → API event intake	Timeouts; retries only for safe reads; write idempotency keys; explicit "ERP unavailable" (never fabricate success); errors mapped to stable application error codes; circuit breaker on repeated failure
Frappe (custom app)	Atomic rules, permissions, scheduler	Adapter → whitelisted methods; Frappe hooks → signed callbacks	Version-compatibility check at startup (adapter ↔ custom app version); rule failures return structured reasons (e.g., full, ineligible, duplicate); deploy new app version before adapter version
Razorpay	Online payments (test mode P1)	API creates order → client checkout → Razorpay webhook to API (signature verified) → payment marked Paid → ERP posting → outbox event	Only verified server-side confirmation marks Paid; idempotent webhook handling; delayed webhook leaves Pending; periodic status-fetch reconciliation for stale Pending; secrets in secret store
WhatsApp Cloud API	Member/lead messaging	n8n `send-message` → WhatsApp; status webhooks → API →	Approved templates only; bounded retry with back-off;

Integration	Purpose	Data Flow	Error Handling
		delivery record	template-rejection logged and surfaced to admin; mock mode until production credentials exist; per-tenant credentials supported by design
Email services (P2)	Receipts, renewals, follow-ups	n8n or API adapter → provider; bounce/complaint webhooks → API	Suppression list honoured; retries; provider abstraction to allow change
AI services	LLM for tool-calling, summarisation, classification	AI module → LLM gateway → provider	Per-call timeout; one bounded retry; graceful degradation ("assistant unavailable") with UI/dashboards unaffected; token/cost limits per tenant and tier; PII minimisation; provider-agnostic interface
Google Sign-In / Phone OTP	Member authentication	Auth module → identity provider / OTP provider	OTP rate limits and expiry; provider failure falls back to retry or alternative method; OTP provider not yet selected
n8n (internal)	Workflow orchestration	Outbox dispatcher ↔ n8n webhooks; n8n → API (authenticated service credentials)	Signed webhooks; dispatcher retries with back-off and dead-letter state; n8n outage does not block transactions (queued)

---

12. Security Architecture

12.1 Authentication

Population	Method	Notes
Staff	Email + password; mandatory reset; MFA (TOTP) for Owner/Admin before production	Password hashing with a modern adaptive algorithm; lockout and rate limiting
Members	Phone OTP primary; optional Google sign-in	Account linking on verified phone; never creates duplicate member
Visitors	None	Public routes only; bot protection on forms and chat
Services (n8n, ERP callbacks, agents)	Dedicated service credentials/signed requests	Scoped, rotated, non-interactive
Sessions	Short-lived access token + refresh rotation; HTTP-only secure cookies	Tenant claim bound to host

12.2 Authorization and RBAC

Evaluation chain per request: **Identity → Role → Permission → Tenant/Studio/Branch scope → Resource ownership.**

- Roles are bundles of granular permissions stored in the App DB (Owner, Admin, Front Desk, Trainer, Member, Visitor, plus operator-side Platform Administrator).
- Guards call a central policy service; code never checks `role === "admin"` inline.
- Special grants (e.g., trainer-assisted booking) are explicit, scoped and revocable.
- Trainers restricted to assigned sessions; Members to own records.
- UI hides disallowed actions, but only the server decides.
- AI tools list is filtered per user and re-checked at execution.
- Business rules that no role can bypass (eligibility, payment, capacity) sit in the ERP transaction.

12.3 Tenant Security

Tenant guard + RLS + ERP tenant permission hooks + tenant-scoped AI retrieval + per-tenant credentials and templates (see §6.1). Cross-tenant attempts return not-found style responses, are audit-logged and alert on repetition. Platform Administrator access to tenant data occurs only through audited support procedures with time-boxed access.

12.4 API Security

- HTTPS everywhere; HSTS; security headers; strict CORS allow-list.
- Schema validation on all inputs; request-size limits; per-IP/per-user/per-tenant rate limits; stricter limits for public, OTP and AI endpoints.
- Webhooks (Razorpay, WhatsApp, ERP, n8n): signature verification, timestamp/replay window, idempotency.
- Secrets in a secret store/environment injection; never in source control or client code; write-only in the UI.
- ERP accessed only by restricted integration identity from the API layer; ERP not exposed to browsers.
- Payment state changes only from server-verified gateway confirmations.
- Encryption in transit and at rest; PII minimised in logs and prompts; dependency and container scanning in CI.
- Data-protection alignment: consent, purpose limitation, retention and deletion handling (legal review pending).

12.5 Audit Logging

Aspect	Design
What	Approvals, payment changes, attendance corrections, role/permission/config changes, deletions, tenant-boundary denials, every AI tool call, every automated send
Record	Tenant, actor, actor type (human / automation / AI), action, resource, before/after, timestamp, request context, correlation ID; AI adds request, tool, arguments, decision, approval, result
Write path	Interceptor/service writes in the same transaction as the outbox event; append-only table (no update/delete permission for the app role)
Access	Owner (full), Admin (limited); read-only views; export in Phase 2
Correlation	One ID propagated UI → API → ERP → n8n → WhatsApp callbacks
Retention	Per tenant policy and legal review; critical history archived, not deleted

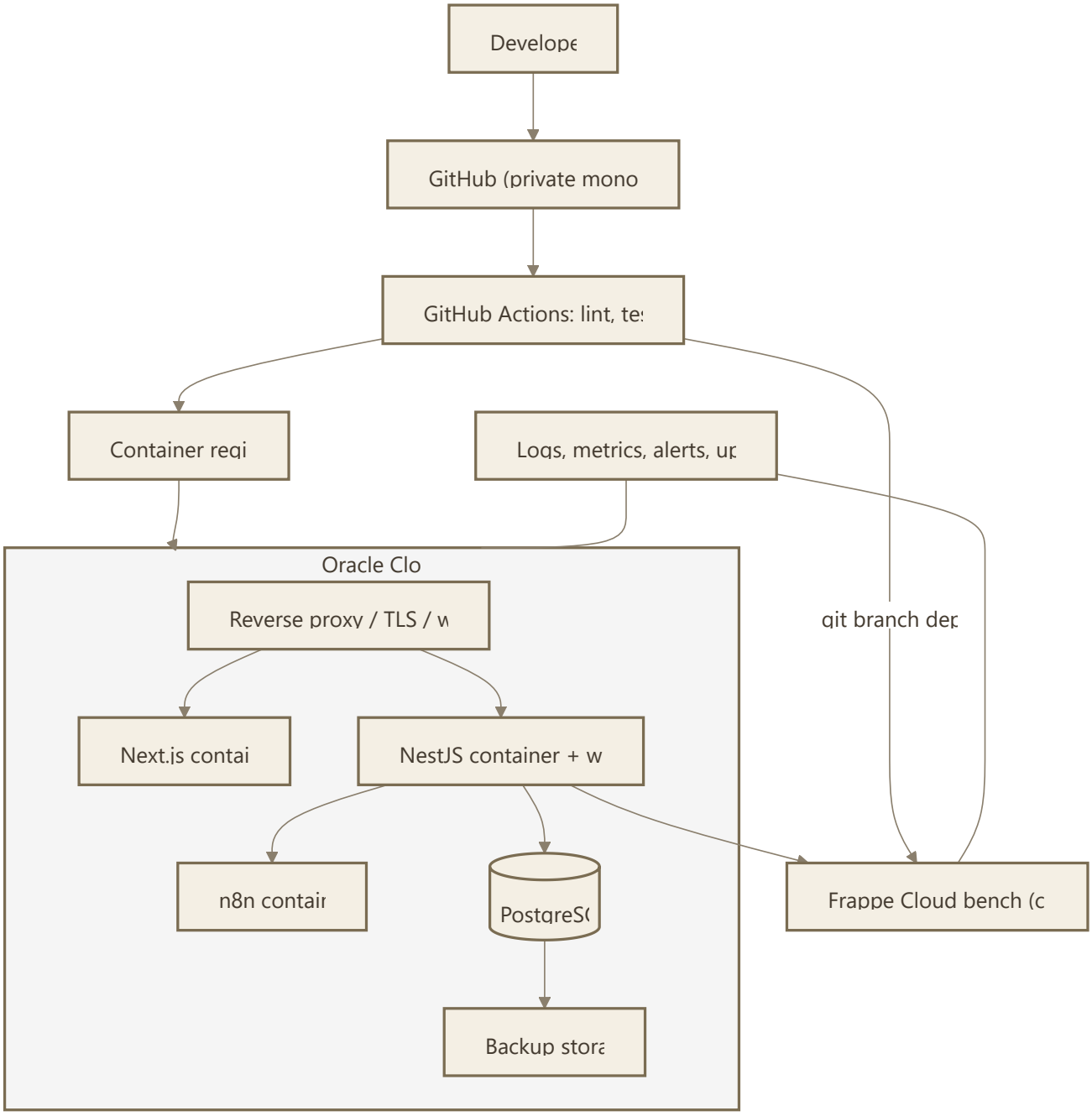
13. Deployment Architecture

13.1 Environments

	Development	Testing (staging / UAT)	Production
Purpose	Feature work, integration	Regression, tenant-isolation and AI evaluation, demo	Live tenants
Web + API	Local Docker Compose (lightweight) plus shared dev host on Oracle	Oracle Cloud VM (containers)	Oracle Cloud (containers, reverse proxy, TLS, load-balanced when needed)
PostgreSQL	Local container	Separate instance, seeded demo data	Managed/backed-up instance with PITR, later read replica
ERP	Shared **Frappe Cloud dev site** (avoid heavy local ERP runtimes)	Frappe Cloud staging site	Frappe Cloud production site
n8n	Shared dev instance	Staging instance	Production instance (isolated credentials, backups)
WhatsApp / Razorpay	Mock / test	Mock / test	Live credentials (Phase 2)
LLM	Dev key, low limits	Test key	Production key, per-tenant quotas
Data	Synthetic	Anonymised/synthetic demo	Real
Secrets	`.env` (untracked) / dev vault	Secret store	Secret store, rotated

5-day reality: stand up Development and one shared **Beta/Staging** environment (doubles as the demo environment) in Phase 1. Production is provisioned in Phase 2 alongside production credentials, MFA enforcement and the security review — architecture and pipelines are identical so promotion is a configuration change.

13.2 Infrastructure Components



Component	Notes
Reverse proxy / TLS	Wildcard certificate for tenant subdomains; only 80/443 public
Worker	Same NestJS image, worker entry: outbox dispatcher, schedulers, async AI jobs
n8n	Private access to API; webhook endpoints protected by signature; database persisted and backed up
Observability	Structured JSON logs with correlation ID and tenant; error tracking; uptime and synthetic checks for canonical flow; alerts on failed workflows, webhook errors, outbox backlog
Backups	Daily PostgreSQL backups with tested restore; Frappe Cloud backups for ERP; n8n data backup; rollback plan per release

13.3 Deployment Flow

1. Feature branch → pull request → CI: lint, type-check, unit tests, API tests, dependency/secret scan.

2. Merge to `develop` → auto-deploy to Development (containers; Frappe custom app branch auto-updated on dev site).
3. Merge to `release` → deploy to Testing: run migrations, run E2E canonical flow, tenant-isolation suite, AI evaluation set; UAT.
4. Approved tag → Production: ordered rollout — **(a) Frappe custom app, (b) database migrations (backward-compatible), (c) API/worker, (d) web, (e) n8n workflow import** — followed by smoke tests (login, member, booking, payment test, notification) and monitoring window.
5. Rollback: redeploy previous container tags; Frappe Cloud app rollback; backward-compatible migrations only (expand → migrate → contract).

13.4 Environment Strategy

- Same containers and configuration schema in every environment; differences only via environment variables.
- Feature flags gate AI and unfinished capabilities per tenant/tier.
- Mock/live switches for WhatsApp and payment are configuration, not code.
- Custom app and API versions are compatibility-checked at startup.
- No secrets, dependency folders, virtual environments or database backups in Git.

---

14. Project Structure

14.1 Repository Structure (private monorepo)

```
wellness-studio-os/
├── apps/
│   ├── web/                # Next.js: staff + member + public
│   ├── api/                # NestJS modular monolith
│   └── worker/             # Outbox dispatcher, schedulers, async AI jobs
├── packages/
│   ├── contracts/          # OpenAPI spec + generated types/clients
│   ├── ui/                 # Shared design system
│   ├── config/             # Lint, TS, shared config
│   └── ai-tools/           # Tool definitions shared by API and evaluation harness
├── erp/
│   └── wellness_studio/    # Custom Frappe app (ERPNext stays upstream, not vendored)
├── automation/
│   └── n8n/                # Workflow exports + deploy scripts
├── infrastructure/         # Docker, compose, proxy, CI/CD, environment templates
├── docs/                   # Architecture, decisions (ADRs), runbooks, test evidence
├── scripts/
└── .github/workflows/
```

14.2 Frontend Structure

```
apps/web/
├── app/
│   ├── (public)/           # Landing, classes, events, enquiry
│   ├── (auth)/             # Staff login, member OTP, reset
│   ├── (staff)/            # B2B portal routes
│   └── (member)/           # B2C portal routes
├── features/               # One folder per domain: members, memberships, classes,
│                           # bookings, attendance, crm, events, payments, reports,
│                           # staff, settings, notifications, ai
├── components/             # ui, layout, tables, forms, feedback states
```

```
├── lib/                    # api client, auth, permissions, tenant theme, validation
└── middleware.ts           # Host→tenant resolution, route protection
```

14.3 Backend Structure

```
apps/api/src/
├── common/                 # guards, interceptors, filters, pipes, errors, context
├── auth/  tenants/  users/  roles/
├── members/  memberships/  classes/  bookings/  attendance/
├── events/  payments/  crm/  reports/  staff/  settings/
├── notifications/          # templates, delivery records, outbox, dispatcher
├── audit/
├── integrations/           # erp/  razorpay/  whatsapp/  email/  llm/
├── ai/                     # gateway/  orchestrator/  tools/  guardrails/  pending-actions/
└── database/               # migrations, seeds
```

Each domain module: controller · service · policy · repository · DTO · tests.

14.4 ERP Structure

```
erp/wellness_studio/
├── hooks.py                # doc_events, scheduler, permission hooks
├── api/                    # whitelisted methods (atomic operations)
├── doctypes/               # custom DocTypes
├── rules/                  # eligibility, capacity, state transitions
├── permissions/            # tenant scoping
├── events/                 # signed callbacks to API
├── patches/  fixtures/     # migrations, seed configuration
└── tests/
```

14.5 AI Structure

```
apps/api/src/ai/
├── gateway/                # provider adapters, retries, cost limits
├── orchestrator/           # conversation loop, confirmation state machine
├── tools/                  # thin wrappers over domain services (per agent/assistant)
├── assistants/             # admin, enquiry, booking, recommendation, feedback
├── agents/                 # (Phase 2+) crm, operations, marketing, ...
├── guardrails/             # policies, redaction, scope rules
└── evaluation/             # curated query sets, scenario tests, recorded results
```

14.6 Automation Structure

```
automation/n8n/
├── workflows/
│   ├── lead-confirmation.json
│   ├── booking-confirmation.json
│   ├── class-reminders.json
│   ├── renewal-reminders.json
│   ├── payment-received.json
│   ├── event-registration.json
│   ├── send-message.json           # shared sub-workflow: mock/live, retry, status
│   └── failure-alert.json
├── credentials.template.md         # names only, no secrets
└── deploy/                         # import/export scripts
```

14.7 Team Ownership (6 members)

The sources name four team members (Tahir, Harish, Shreya, Harshini). Members 5 and 6 are **not named or described anywhere in the provided documents**, so they are shown as placeholders; the split below is a proposal for the team lead to confirm.

Member	Primary ownership	Repository areas
Tahir	Solution lead: API layer foundations (auth, tenancy, RBAC, audit, outbox), ERP adapter, integration boundary, product-to-backlog translation, security rules	`apps/api/common`, `auth`, `tenants`, `audit`, `integrations/erp`, `packages/contracts`, `docs`
Harish	Domain modules (members, memberships, classes, bookings, attendance), API testing, n8n workflows and WhatsApp mock/live path	`apps/api/{members,memberships,classes,bookings,attendance}`, `automation/n8n`, `apps/worker`
Shreya	AI platform and agents: LLM gateway, tool registry, guardrails, Enquiry / Booking / Recommendation / Feedback assistants, future agent design, evaluation harness	`apps/api/ai/{gateway,tools,guardrails,agents,evaluation}`, `packages/ai-tools`
Harshini	Admin AI Assistant: orchestrator conversation loop, Create Member flow, pending-action confirmation, query set, assistant tests and documentation	`apps/api/ai/{orchestrator,assistants/admin}`, staff chat panel in `apps/web/features/ai`
Member 5 (name TBC)	ERP/Frappe: custom app, DocTypes, atomic booking/check-in methods, tenant permission hooks, Frappe Cloud bench and environments	`erp/wellness_studio`, Frappe Cloud config
Member 6 (name TBC)	Frontend: B2B and B2C portals, design system, tenant theming, client-	`apps/web`, `packages/ui`

Member	Primary ownership	Repository areas
	side states; supports payments (Razorpay checkout), events, payments and reports modules	

Shared responsibilities: CI/CD and Oracle Cloud environments (Tahir with Harish), QA and E2E canonical-flow testing (all, coordinated by Tahir), documentation (each owner for their area). Every feature follows the PRD sequence: Business Requirement → Full-Stack Capability → API/Tool → AI/Agent Capability → Automation → End-to-End Validation, with the relevant owners collaborating on each.

Appendix — Architecture Risks and Validation Spikes (Day 1)

#	Item	Why	Exit criterion
S1	Frappe Cloud custom-app deployment from private repo	Risk R-14	Custom app installed and updated from GitHub on dev site
S2	ERP tenant isolation approach (Company + permission hooks vs. site-per-tenant)	PRD leaves mechanism open	Cross-tenant read/write denied in a two-tenant test
S3	Atomic booking in ERP under concurrency	NFR-PRF-03	50 parallel last-seat requests → exactly one confirmation
S4	ERP adapter latency budget	Extra hop (R-03)	Booking p95 under 1 second end-to-end on beta environment
S5	Outbox → n8n → WhatsApp mock end-to-end with dedup	Automation reliability	Duplicate event delivery produces one message
S6	LLM tool-calling with propose/confirm	AI safety goals	Create Member scenarios: missing fields, duplicate, permission denied, API failure — no false success
S7	Phone OTP provider selection	Not specified in sources	Provider chosen and sandbox verified